IACR Transactions on Cryptographic Hardware and Embedded Systems
ISSN 2569-2925, Vol. 2025, No. 3, pp. 115–138. DOI:10.46586/tches.v2025.i3.115-138

TREE: Bridging the gap between reconfigurable computing and secure execution

Sérgio Pereira[1], Tiago Gomes[2], Jorge Cabral[3] and Sandro Pinto[4]

[1] Centro ALGORITMI/LASI, Universidade do Minho, Guimarães, Portugal,
sergio.pereira@dei.uminho.pt
[2] mr.gomes@dei.uminho.pt
[3] jcabral@dei.uminho.pt
[4] sandro.pinto@dei.uminho.pt

Abstract. Trusted Execution Environments (TEEs) have become a pivotal technology for securing a wide spectrum of security-sensitive applications. With modern computing systems shifting to heterogeneous architectures, integrating TEE support into these systems is paramount. One promising line of research has proposed leveraging FPGA technology to provide promising TEE solutions. Despite their potential, current implementations of FPGA-based TEEs have a set of drawbacks. Some solutions (i.e., MeetGo and ShEF) prioritize the secure loading of reconfigurable modules but lack compatibility with established legacy TEE specifications and services. On the other hand, those that aim to establish legacy compatibility (i.e., TEEOD and BYOTee) fail to fully utilize the dynamic reconfigurability and parallel processing capabilities inherent in FPGAs. In this context, we introduce Trusted Reconfigurable Execution Environments (TREE), a novel framework that fulfills the gaps existing in current FPGA-based TEE approaches. TREE enables system designers to fully leverage the reconfigurability capabilities of FPGAs without compromising compatibility with existing TEE specifications. Our reference TREE implementation ensures secure execution of user-customized hardware, legacy software trusted applications (TAs), and TAs that combine both custom hardware and software components, by fully exploiting the FPGA's dynamic partial reconfiguration capabilities. TREE's root of trust relies on conventional SoC-FPGA mechanisms including secure initial reconfiguration and memory protection, to ensure the initial bitstream integrity is kept after loaded and that reconfiguration access is restricted to the FPGA fabric after boot. Additionally, TREE provides essential TEE services within the FPGA fabric, including secure storage and cryptographic functions, enabling TAs to securely store sensitive data and perform critical operations in an isolated environment. Our evaluation on an entry-level FPGA, involved assessing TREE using microbenchmarks and real-world applications to compare its hardware costs and performance speedups against OP-TEE. The results showed that TREE's hardware costs are minimal, while it achieves significant performance speedups, especially when compared to hardware TAs. For empirical demonstrations, we assess two real-world TA examples on TREE: an access control authenticator and a Bitcoin wallet.

Keywords: Trusted execution environment; Field programmable gate arrays; Reconfigurable logic; FPGA; Dynamic Partial Reconfiguration (DPR); Secure storage; Cryptography; Attestation; Information technology;

1 Introduction

As modern computing systems transition from CPU-centric designs to heterogeneous computing, the security implications associated with these new architectures have not

Received: 2025-01-15 Accepted: 2025-03-15 Published: 2025-06-05

yet been met with adequate solutions [CCF+16, EGEAH+08, NLSY19]. Heterogeneous architectures (also known as xPU) combine different processing units, such as CPUs, graphics processing units (GPUs), Field Programmable Logic Arrays (FPGAs), and domain-specific accelerators (DSAs), to meet the growing computational needs and energy efficiency. Despite the performance benefits and the security measures in place for each unit individually, xPU architectures face unattended challenges in security [YFW18, ZHW+20]. For example, adversaries can exploit one unit's specific vulnerabilities to compromise others and potentially compromise data confidentiality and integrity across the system [XLXW21].

Trusted Execution Environments (TEEs) have become a pivotal and continuously evolving technology for protecting security-sensitive applications [CSFP20]. Available on most commercial off-the-shelf (COTS) platforms, TEEs isolate the execution of trusted applications (TAs) from the untrusted main operating system, or Rich Execution Environment (REE) [SAB15]. Moreover, to standardize and enhance TA development, the GlobalPlatform [Glo24] TEE specifications are widely adopted. Intel SGX [Int23], Arm TrustZone [Arm09], and AMD SEV [AMD19] are well-established TEE technologies designed primarily for CPU-centric architectures.

Unfortunately, extending TEE support to xPU architectures poses several challenges. High-throughput xPUs (e.g., GPUs and FPGAs) achieve high performance by integrating multiple cores and using high-bandwidth memory. Adding features like memory confidentiality and integrity via encryption engines or Merkle Trees can reduce available memory capacity and bandwidth, critical for these xPUs [SCG+03]. Similarly, enforcing memory isolation through SGX-like checks during address translation would severely under-utilize accelerators due to their sensitivity to address translation latency [PPM15].

With the rise of general-purpose GPUs, several works have successfully implemented TEEs on GPUs, with minimal or no hardware modifications required [VVB18, JTK+19, LGL+21, DWY+22]. These initiatives have paved the way for enhanced security guarantees on these systems. However, the security landscape differs significantly for other high-throughput xPUs, such as FPGAs. Unlike GPUs, which are tightly controlled by CPUs, FPGAs operate independently as hardware accelerators [TM14]. This introduces potential risks as CPU vulnerabilities could compromise the integrity of the reconfigurable logic in FPGA systems-on-chips (SoC-FPGA). Adversaries exploiting a compromised CPU could manipulate FPGA operations or access sensitive data [BMAB17, JHZ+17].

Researchers have explored the challenges of implementing isolated execution environments on SoC-FPGAs to protect sensitive operations from unauthorized access and tampering. ShEF [ZGK22] and MeetGo [ONJ+21] have taken initial steps towards FPGA-based TEEs by relying on conventional FPGA mechanisms to secure loading custom hardware designs onto Cloud's FPGAs. Other works (i.e., TEEOD [PCRP21] and BYOTee [AZ24]) focus on isolating sensitive applications from malicious software, or hardware logic. This second approach enables TEEs by synthesizing soft microcontroller units (MCUs) at start-up, providing dedicated execution environments for software TAs.

Despite the progress made by these FPGA-based TEE solutions, none fully support unified TA management from development through runtime. MeetGo and ShEF are tailored for hardware-focused applications and lack support for software and GlobalPlatform-compliant TAs. While TEEOD and BYOTee address this gap, they fall short on leveraging FPGA's partial reconfigurability. They rely on full system reconfiguration for new TA hardware requirements, rather than supporting independent isolation, updating, or optimization of TAs through partial reconfigurability. As a result, these post-boot fixed environments may struggle to accommodate unexpected runtime workloads, such as increased data processing (e.g., machine learning inference or real-time analytics) or adapting to evolving cryptographic algorithms (e.g., transitioning to new digital signature algorithms).

This paper builds on existing FPGA-based TEE approaches, advocating for the adoption of Trusted Reconfigurable Execution Environments (TREE). The TREE framework

allows system designers to synthesize (or compile) and execute customized TAs—whether hardware-based, software-based, or hybrid—isolated from the REE. Similar to other FPGA-based TEEs, TREE's secure system resides in the FPGA fabric, while the remaining SoC units belong to the REE. TREE stands out by leveraging dynamic partial reconfiguration (DPR) technology, prevalent in most SoC-FPGAs, to facilitate the loading of TAs' hardware, while ensuring their integrity. Additionally, TREE's root of trust relies on standard SoC-FPGA mechanisms, such as secure reconfiguration and memory protection, removing the need for specialized hardware or proprietary extensions. TREE is designed to integrate seamlessly into existing development workflows, providing tools and processes that simplify TA development and deployment. It also offers TEE services like secure storage and cryptography, compliant with GlobalPlatform specifications.

Our evaluation of TREE was performed on an AMD Zynq UltraScale+ MPSoC development board (Ultra96-V2), featuring an entry-level SoC-FPGA. Our analysis of the reconfigurable hardware costs showed that TREE requires minimal FPGA logic elements, even on a low-resource SoC-FPGA, leaving ample capacity for TA hardware. To assess different TA deployment models, we compared TREE with the vanilla OP-TEE implementation, a widely used GlobalPlatform-compliant TEE on Arm TrustZone. Results show that TREE achieves faster round-trip times in GlobalPlatform-compliant CA operations (e.g., session opening, closing, and command invocation). TREE's TAs outperform OP-TEE's in basic computational tasks—such as checks and secure storage accesses. For more complex tasks, including cryptographic operations, TREE's TAs show notable speedups over OP-TEE's standard TAs. However, software-based TAs experience a slight slowdown, justified by the use of a small soft MCU. We demonstrate TREE's real-world applicability with two proof-of-concept applications: an access control authenticator and an open-source, hardware-based Bitcoin wallet.

In summary, with TREE, we make the following **contributions**:

1. **Conceptualization of TREE**: Introduce the concept of TREE to address the limitations of current FPGA-based TEEs, focusing on dynamic reconfigurability.

2. **TREE Framework Design**: Develop a comprehensive security infrastructure within the TREE framework to ensure integrity and confidentiality while providing access to essential TEE features, such as secure storage and cryptographic engines, from the boot process onwards.

3. **Open-Source Reference Implementation**: Provide an open-source reference implementation of the TREE framework, facilitating adoption and further development by the research community and industry practitioners.

4. **Comprehensive Evaluation and Benchmarking**: Conduct thorough evaluation and benchmarking of the TREE framework to demonstrate its security, performance, and flexibility compared to existing solutions.

5. **Empirical Evaluation of TREE in Practical Applications**. To demonstrate TREE's practical applicability, we evaluate its performance in real-world scenarios through two case studies: an access control authenticator and a Bitcoin wallet.

2 Background and Overview

2.1 FPGA-SoC Partial Reconfiguration

SoC-FPGAs, e.g., AMD Zynq Ultrascale+ [AMD24], integrate a programmable logic (PL) and a processing system (PS) within the same die. The PS encompasses non-reconfigurable units, i.e., application processor units (APUs), platform management

Table 1: Architectural Features of Commercial FPGAs Supporting DPR technology.

Architecture family	PR Granuality	PR Primitive
AMD Virtex-5/6/7	One clock region high	ICAP
AMD Zynq	One clock region high	ICAP/PCAP
AMD Ultrascale	One CLB	ICAP/MCAP
Altera Stratix-V/Arria-10	One ALM	PR-IP
Altera Stratix-10	One sector	SDM

units (PMUs), and configuration security units (CSUs), which handle processing tasks, system management, and security operations, respectively [Tay17]. The PL's functionality is defined by bitstreams generated from hardware designs synthesized using hardware description languages (HDL), e.g., Verilog or VHDL [Tri94]. FPGAs support various applications, including flexible software execution via soft microcontrollers (soft MCUs) within their fabric [TM14, Xil24].

Dynamic partial reconfiguration. SoC-FPGAs, e.g., AMD Zynq, enable partial reconfiguration of the PL during system initialization through various ports: i) the internal configuration access port (ICAP) for self-reconfiguration, ii) the processor configuration access port (PCAP) from the PS, and iii) the joint test action group (JTAG) interface for external configuration and debugging [Pet24]. In applications requiring new functionality or feature updates, some FPGAs provide DPR technology, i.e., the ability to reprogram specific regions of the fabric, known as partial reconfigurable regions (PRRs), on the fly without disrupting the rest of the device [VF18, LFy09]. Table 1 compares the architectural features of commercially available FPGAs supporting DPR.

Reconfiguration access control. Under normal conditions, the PS has full control over the PL, allowing the PS to reconfigure, read data, clear, or power down either the entire PL or the PRRs. However, this control can be restricted to prevent unauthorized access. Manufacturers have implemented security measures to establish a loose coupling between the PL and the PS. These include having PCAP and ICAP interfaces mutually exclusive by design, disabling reset and readback ports, enabling encryption-only reconfigurations, implementing memory protection units (MPUs), enabling secure boot processes, and using physically unclonable functions (PUFs). These standard security features may serve as a strong foundation but could fall short in addressing scenarios where different security domains must co-exist within the PL. In such cases, additional security techniques (e.g., fine-grained isolation using TEEs or hardware-based memory partitioning) must be employed to establish isolated and independent regions within the PL [RMIH23].

2.2 Trusted execution environments

TEEs protect the processing of sensitive data, such as biometric data or cryptographic keys, in an untrusted environment. These secure environments comprise hardware and software features and the trusted computing base (TCB) [SAB15]. The GlobalPlatform TEE Specification [Glo18], widely adopted by the academia and industry and supported by numerous manufacturers and software developers, outlines the key components of a generic GP-compliant TEE (as shown in Figure 1). The Regular Execution Environment (REE), which runs outside the TEE and contains non-trusted applications and services, interacts with the TEE through the GP TEE Client API. Key trusted components include the TEE OS, which consists of a secure kernel, a trusted user interface, and secure services; TAs, which are the applications that execute secure-sensitive operations on top of the TEE OS and take leverage of the secure services provided by the GP TEE internal core API; and GP-compliant client applications (CAs), applications that run outside the TEE and use the GP TEE Client API to communicate with TAs [Glo24].

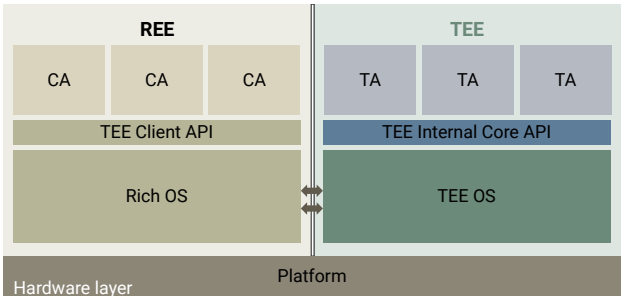


Figure 1: Generic TEE components complying with the GlobalPlatform specification. The REE, shown in brown, contains the Rich OS (e.g., Linux) providing non-trusted services, and CAs that interact with the TEE. The TEE, highlighted in green and blue, includes trusted components, i.e., the TEE OS and TAs. The TEE Client API and TEE Internal Core API facilitate secure communication and provide TEE services, respectively.

TEEs in xPUs. With the shift from CPU-centric designs to heterogeneous computing architectures, TEEs now face the challenge of isolating trusted applications from malicious operating systems and malicious hardware and software across various processing units (xPUs), such as GPUs and TPUs [ZHW+20, XLXW21, MRRL23]. This shift brings additional complexity to the design and implementation of TEEs. Additionally, TAs increasingly leverage these heterogeneous architectures to improve performance, utilizing specialized processors to accelerate tasks like cryptographic operations [HBS+08].

3 Threat Model & Design Goals

3.1 Threat Model

We consider a powerful adversary whose primary goal is to extract sensitive information from the TEE or a compromised TA, either locally or remotely, without resorting to intrusive methods. Such an adversary is capable of controlling and compromising all REE software and hardware components, such as the OS, memory, storage, and the platform's system interconnect, including the interfaces connecting to the TEE hosted on the FPGA fabric. The adversary might attempt to load, read back, or tamper with the hardware designs loaded on the FPGA fabric. However, it is assumed that access to the FPGA is attempted through standard operations (i.e., loading, reading back, or modifying via conventional channels) rather than through intrusive physical tampering.

Additionally, we assume that the development framework, compilers, and synthesis tools are trustworthy. We assume the SoC has not been subjected to tampering, thereby preserving the integrity of its hardware resources and standard FPGA-SoC functionalities (e.g., secure boot and reconfiguration access control), as guaranteed by the manufacturer.

FPGA-SoC specific threats. Our threat model adheres to the one commonly assumed for TEE architectures, e.g., as outlined by Cerdeira et al. [CSFP20], while accounting for the unique characteristics and known vulnerabilities of FPGA-SoC platforms. Specifically, it considers potential attacks on the dynamic partial reconfiguration process, where an adversary might attempt to inject malicious bitstreams and software applications. We also account for the risk of a TA being compromised, and an adversary potentially escalating access and privileges to gain unauthorized control over other TAs or TEE resources. Intrusive physical attacks (e.g., micro-probing or fault injection) and side-channel attacks (e.g., power analysis or electromagnetic leakage) are out of scope, as their mitigations are orthogonal and can be integrated independently (see section 9 for further details).

3.2 Design Requirements

Considering the aforementioned threat model, we resorted to a set of principles to guide our design. We propose introducing the concept of reconfigurable TEEs. Reconfigurable TEEs address the shortcomings observed in existing FPGA-based TEEs by focusing on two key aspects: dynamic reconfigurability and comprehensive security features. The goal is to create a flexible framework for SoC-FPGAs, ensuring the integrity and confidentiality of various security-sensitive applications within the PL. To effectively achieve this goal, we propose the following design requirements:

R1: Isolate FPGA fabric from the REE. Create an isolated environment within the FPGA fabric to prevent unauthorized access and interference from the rest of the SoC-FPGA and to prevent malicious, non-secure software or hardware components from accessing or tampering with the FPGA fabric logic and the TAs. This principle aligns with resource isolation guidelines in secure system design [SWGI+16], which advocate avoiding shared hardware and software resources to prevent compromise and manipulation. As found in the literature [TSS17, LSP+17, K. 19, Car], shared resources have repeatedly been leveraged to extract the secrets and manipulate TA functionality.

R2: Isolate each TA execution environment. Establish dedicated and distinct secure environments for each TA, ensuring minimal sharing of FPGA logic elements to prevent potential interference and maintain integrity and confidentiality. Hardware TAs require dedicated FPGA resources such as logic elements and memory spaces, while software TAs require dedicated physical address spaces, computing units, and peripherals. Implement an efficient clean-up process to facilitate the reuse of secure environments for hosting different TAs during runtime. This approach supports the least privilege principle by limiting each TA to only the necessary for its operation, thus reducing the attack surface [Ben16].

R3: Ensure TA sensitive data protection. TREE must provide secure storage primitives to protect sensitive data from unauthorized access, tampering, or leakage. This includes access control to guarantee only authorized TAs can access the stored data. This requirement aligns with guidelines to protect sensitive information through strict access controls and encryption [CFZ+20, Glo18, Glo24].

R4: Enable versatile TA synthesis and compilation. Enable TA developers to design and compile custom hardware TAs, software-based TAs (including legacy TAs), and hybrid TAs (software TAs running on specialized hardware). This functionality should include both development-time tools and runtime processes, allowing developers to create and deploy their TA configurations within the FPGA fabric—whether hardware, software-based, or hybrid. This flexibility facilitates the integration of various TA configurations while managing complexity and mitigating potential vulnerabilities [BBD+21].

4 Overview & Architecture

Guided by the threat model and design goals in sections 3.1 and 3.2, the TREE design assumes all software running on the PS and associated hardware components as insecure, belonging to the REE, while the FPGA fabric is considered trusted. TREE divides the FPGA fabric into two modules: the **TrustShell** and **Dynamic Tiles**. The TrustShell incorporates several key elements that ensure secure operation and communication within the FPGA. Dynamic Tiles are the different dedicated execution environments for TAs and their exclusive interfaces to the TrustShell. Figure 2 illustrates TREE's core design elements, including the REE, TrustShell, and Dynamic tiles.

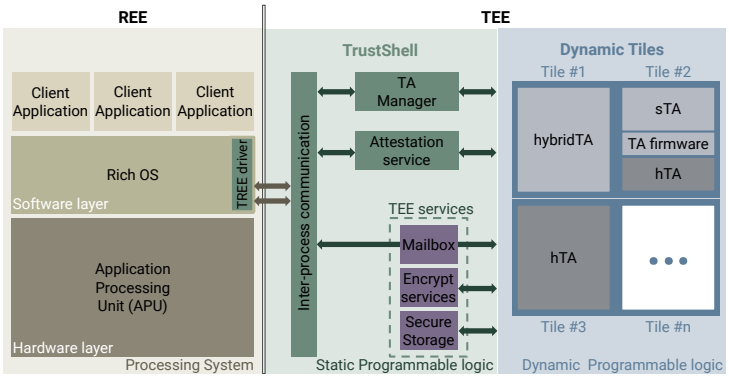


Figure 2: Design Elements of the Trusted Reconfigurable Environment (TREE) within an SoC-FPGA.

4.1 TrustShell

Configured securely into the FPGA during boot, the TrustShell corresponds to the TREE's static portion and is responsible for: i) enabling interprocess communication (IPC) between the CAs and TAs; ii) managing, loading, and attesting TAs; and iii) providing TEE services to TAs. The main components of TrustShell are:

Tile manager engine. This engine handles loading and managing available Tiles. It verifies the integrity and authenticity of each TA before loading it into a dynamic tile. The engine manages the full lifecycle of TAs, including loading, execution, and cleanup. Additionally, it provides to the TEE services information about the current TAs in execution, removing the need to rely on the TAs themselves for identification.

TrustShell IPC. This mediator manages the communication between the REE and TAs within dynamic tiles. It parses incoming REE messages, updates the Tile Manager Engine, and forwards them to the respective TA upon authorization.

Attestation service. This service verifies the integrity of the TrustShell configuration and active TAs, ensuring that the static configuration and TAs remain genuine and untampered.

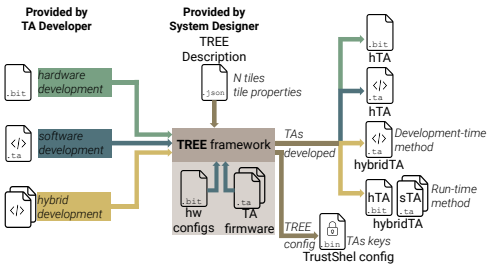
TEE services and secure storage. TrustShell provides TAs with a range of TEE services, including secure storage, cryptographic operations, and time services.

4.2 Dynamic tiles

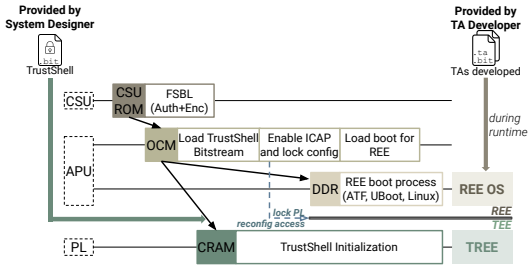
Each tile executes a single TA at a time and has exclusive interfaces to the TrustShell, shared memory, and TEE services. To support the execution of multiple TAs concurrently, additional dynamic tiles must be configured, with corresponding interfaces replicated for each TA. This architecture ensures independent and simultaneous TA execution. The main components of dynamic tiles include:

Partial reconfigurable regions. Each dynamic tile contains a PRR, a region within the FPGA fabric where a TA is loaded and executed. These regions can be dynamically reconfigured to load different TAs, enabling the execution of various hardware designs.

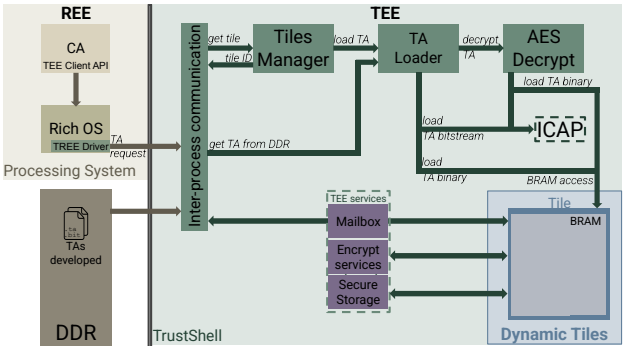
TA firmware. To support software TAs, compatible soft computing hardware must be configured within the PRR before loading the TA. The firmware, loaded with the TA, provides the runtime environment for execution, including essential kernel functions for booting, message parsing, error handling, and communication with TEE services.



(a) hTAs, sTAs and hybridTAs development differences using the TREE framework.



(b) Boot-time sequence to configure the TrustShell and restrict the REE from accessing the PL.



(c) TREE runtime modules to manage, load, and monitor TAs and provide TEE services to them.

Figure 3: Workflow of the TREE framework at development-time (a), boot-time (b), and runtime (c). During development, the TREE framework allows system designers to synthesize/compile custom hTAs, software-based TAs, and hybrid TAs. During boot, the TrustShell is loaded to the PL. During runtime, TrustShell facilitates IPC between REE-TAs, manages and attests TAs, and provides TEE services.

5 Design & Implementation

TREE is highly modular and can be configured to provide single or multiple dynamic tiles with varying FPGA fabric allocations of PRRs for each tile. All TrustShell components are synthesized as hardware blocks in the PL. Figure 3 outlines the key differences in the TREE framework across the three processes: i) **development process**, which focuses on the TA development stage, designed to comply with GP specifications and offers similar TA development and TEE-REE interactions as other GP-compliant TEEs; ii) **boot process**, which involves the boot sequence and TREE configuration in the PL, where the secure environment is initialized to ensure proper operation; and iii) **runtime process**, which centers on running and managing TAs within a reconfigurable, secure environment, ensuring that the TREE framework meets its unique requirements.

TREE strives to align with GP specifications but faces unique challenges. These include handling encrypted TAs, proving GP support, and reusing and cleaning up tiles. These challenges are discussed in detail in Section 5.6.

5.1 TA development process

TREE allows system designers to synthesize/compile: i) custom hardware TAs (hTAs); ii) software-based TAs (sTAs); iii) GP-compliant TAs (GPTAs); and iv) hybrid TAs. Its modular design supports multiple tiles, each with its own PRR. System designers require a TREE configuration file that specifies the sizes and locations of these PRRs.

hTAs development. Developing hTAs is straightforward, as tiles are primarily composed of PRRs. System Designers only need to ensure that their designs adhere to timing constraints and fit within the dynamic regions defined by the TREE configuration.

sTA development. sTAs require a compatible hardware execution environment. TREE provides pre-designed trusted hardware configurations, abstracting hardware design complexities. Additionally, TREE offers compatible, GP-compliant firmware that supports kernel functionalities like booting, message parsing, and error handling. This compliance ensures adherence to widely accepted security standards, facilitating Tas' integration into diverse environments without compromising compatibility or security. Alternatively, system designers can create custom hardware, as detailed in the hybrid TA development.

hybridTA development. TAs that integrate both hardware and software, provide system designers flexibility in leveraging FPGA hardware acceleration alongside software functionalities. Depending on the use case, system designers may need either tightly coupled integration for optimal performance or loose coupling for flexibility in updates and third-party involvement. TREE supports two development models: i) *development-time model*: The software and hardware are integrated during development. In this case, TREE treats the hybrid TA as an hTA during runtime. This approach is ideal for tightly coupled systems that do not require post-deployment updates or external contributions; and ii) *runtime model*: The hardware and software are compiled and synthesized separately. During runtime, the hardware is loaded first, followed by the verification and loading of the software. While introducing some performance overhead, this model offers flexibility, allowing the hardware to support various software versions and third-party applications. TREE ensures compatibility by attesting the hardware and verifying software on-the-fly to prevent unauthorized execution.

5.2 Boot-time process

TREE relies on the secure boot to guarantee: i) the integrity and secure loading of TrustShell into the PL; ii) exclusive TrustShell access to the FPGA configuration port (i.e., ICAP); and iii) restricted access to the PL's reconfiguration port (i.e., PCAP and JTAG), including reconfiguration, readback, clearing, and power management. These measures are necessary to prevent PS and external interactions with the reconfigurable logic, thus ensuring a loosely coupled relationship between the PS and PL.

5.3 Runtime process

After TrustShell is configured and booted, it can load TAs on demand. TREE supports one PRR per tile and one TA per tile at a time, adhering to design requirement *R2* (see Section 3.2). For hTAs, or the hardware components of sTAs and hybrid TAs, TrustShell verifies the hardware's integrity and decrypts it if required before partially reconfiguring the selected TA on an available tile. Additionally, for sTAs and hybrid TAs using the *runtime model*, TrustShell's verification process is repeated to now ensure the integrity of the TA binaries before loading them into the tile's memory. Once the TA is loaded, the REE and TEE communication follows a standard RPC model, facilitated by a shared mailbox infrastructure with IPC mediation. TrustShell handles one request at a time per dynamic tile. During the TA's lifecycle, TrustShell: i) provides attestation services to verify the TA's integrity; ii) maintains tile isolation to ensure security; and iii) delivers core TEE services, including secure storage and cryptographic engines.

5.4 Supporting Globalplatform specifications

CA-TA communication. The communication between a CA and a TA follows a default RPC model, where the CA sends a request and waits for a reply from the TA. Each tile has

a mailbox shared with the REE, facilitating this communication. CAs determine which mailbox to use via a TA identifier register. This register, managed by the Tile manager and read-only accessible by the REE, contains the TA identification running in that tile. The mailbox is inactive if the register is zero, indicating no TA is running in that tile. The mailboxes also include fields for operation-specific parameters and return values, ensuring accurate request processing and correct routing of responses to the requesting CA.

TEE services. A TEE usually offers the TAs a set of security-enabling services. Most of them, such as cryptographic operations and time-related services, are only necessary during TA execution and do not require sharing or interaction with other TAs, except for secure storage. As a result, these kinds of services are integrated into the hTA design or as software services embedded in the TA firmware. These services are optional and do not require special access rights beyond what TAs need, removing the need for privilege mode implementation, as it's up to the TA developer to add them.

Secure Storage. Cryptographic operations and time-related services are necessary only during TA execution, while secure storage operates independently of the TA's activity status. Tiles are wiped and reset when the TA is inactive. Secure storage, therefore, is a TEE service external to the tiles, ensuring the persistent storage of sensitive data. This service requires the universally unique identifier (UUID) of TAs attempting to access their data. However, secure storage cannot rely on TAs for self-identification, as the threat model in section 3.1 considers attacks by malicious TAs on others. TREE addresses this by using the Tiles Manager to provide correct TA identification, ensuring that malicious TAs cannot impersonate others and access data.

5.5 TA attestation mechanism

TREE allows CAs to verify the authenticity and integrity of TAs that are executed within the tiles at any time. The TrustShell carries out proof whenever a request for remote attestation exists. For this, the SA employs a static measurement scheme. Whenever the TrustShell loads an application into a tile, it measures (computes the hash of) the bitstream and stores it in a table along with the TA's UUID. Such static measurement is trustworthy because, in the TREE design, TrustShell is the only entity authorized to access the dynamic area where applications are installed.

5.6 Challenges of integrating DPR with TEE

TREE heavily relies on DPR technology to create a flexible FPGA-based TEE. DPR controller and decoupler features enable TREE to instantiate TAs as partial bitstreams at runtime without disrupting the operation of other tiles, the TrustShell, or the REE. However, the DPR capabilities alone are insufficient and limited to provide a TEE. Key challenges include supporting encrypted TAs to ensure confidentiality, efficiently managing the clearing and reuse of tiles to maintain both security and resource availability, and configuring TAs onto any available tile without any interference or resource contention. These challenges require additional mechanisms and strategies to enhance the basic DPR technology to meet the stringent requirements of a secure and efficient TEE environment.

Encrypted TAs. Most SoC-FPGAs only support reconfiguring unencrypted partial bitstreams or encrypted partial bitstreams with the same key as the full bitstream instantiated. This presents a challenge in offering system designers confidential loading capabilities without requiring them to expose their TAs to the REE, or revealing sensitive keys within TrustShell belonging to other system designers. TREE addresses this challenge by synthesizing an AES symmetric decryption IP before accessing the ICAP. This benign man-in-the-middle decrypts and directly forwards the unencrypted partial bitstream to the ICAP. Our implemented mechanism does not require having all the plain partial bitstreams

Table 2: Synthesized results of TrustShell module on the Ultra96-V2.

TrustShell module		LUT (70336)	LUTRAM (28720)	FF (141120)	BRAM (216)	DSP (360)
Tile Manager	DFX Controller	969 (1.4%)	169 (0.6%)	1149 (0.8%)	0	0
	AES decryption IP	2365 (3.3%)	172 (0.6%)	2970 (2.1%)	0	0
	Attestation Tester	90 (0.1%)	33 (0.1%)	318 (0.2%)	0	0
TEE services	Shared Memory	398 (0.5%)	0	371 (0.3%)	2 (0.9%)	0
	Encryption Services	2435 (3.5%)	174 (0.6%)	3260 (2.3%)	0	0
	Secure Storage	406 (0.5%)	0	383 (0.3%)	2 (0.9%)	0
Total		4228 (6.0%)	374 (1.3%)	5191 (3.7%)	4 (1.8%)	0

at once in a trusted storage. Instead, each decrypted frame is sent directly to the ICAP, avoiding the plaintext temporary storage management.

GPTAs support. Regarding sTAs that are Globalplatform-compliant, compatible hardware must be configured to execute them. System designers can use TREE's trusted hardware configurations that support GPTAs to abstract the complexities of hardware design and maintain their legacy. TA runtime environment support, i.e., TA firmware, is also provided; this firmware provides essential trusted kernel functionalities for booting, message parsing, and error handling. It also serves as the TEE IC API's backend, allowing TAs to interact with the TEE services and hardware resources seamlessly.

Tiles reconfiguration and cleanup. Regarding the cleanup process, TREE keeps blanking bitstreams for each tile. Once the former TA context fully concludes, TREE starts overwriting the PRR with the blanking bitstream. Once the partial reconfiguration is complete, the tiles manager erases the TA entry and resets the mailbox. Only then does the TREE attribute the tile to another TA, maintaining the isolation requirement.

Reconfiguring TAs onto any available tile. The size and location of PRRs must be fixed during development; they cannot be modified without fully reconfiguring the PL. Also, partial bitstreams are tied to specific PRRs, preventing them from being reassigned to different PRRs. TREE tackles this challenge by ensuring all tiles have similar sizes and the hTAs are synthesized for all tiles available. At runtime, TREE selects the suitable PR associated with the available tile, preventing resource allocation issues where multiple TAs demand the same PRR concurrently while others remain unused.

6 Evaluation

We evaluated TREE using the Ultra96-V2[1] development board. TREE heavily relies on DPR technology. The Ultra96-V2 supports dynamic function exchange (DFX), a DPR technology for AMD FPGAs, allowing up to 32 PRRs. We decided only to consider one PRR per tile configured. All FPGA modules were synthesized using the AMD Vivado tool, configured to operate at 100 MHz.

6.1 Reconfigurable hardware costs

We evaluate TREE's FPGA logic resource allocation in single-tile and multi-tile configurations. This serves two goals: i) to assess the minimal overhead by measuring the FPGA logic allocation efficiency of the minimal configuration; and ii) to evaluate scalability by analyzing how resource usage increases with additional tiles.

[1] AMD Zynq UltraScale+ MPSoC ZU3EG, featuring an entry-level FPGA, a quad-core Arm Cortex-A53, and 2GB of DDR4 SDRAM.

Methodology. For each TREE configuration evaluated, we measured the FPGA logic elements required, including look-up tables (LUTs), LUTRAMs, flip-flops (FFs), Digital Signal Processors (DSPs), and Block RAMs (BRAMs). We conducted resource utilization tests to quantify the logic elements used by the static portion of TREE (TrustShell), which includes the Tile Manager, TA loader mechanism, TA decryption engine, and TEE services which include mailboxes for communication between CA-TAs, cryptographic engines, and secure storage. Each configuration considers 8 kiB of secure storage per tile of BRAM. Tile granularity is not relevant to this evaluation, as it does not belong to TrustShell.

Single-Tile configuration. Table 2 details the FPGA logic elements usage of each TrustShell component for the single-tile configuration. The Tile Manager, composed mainly of combinational logic, consumes 4.8% of LUTs, 1.3% of LUTRAM, and 3.1% of FFs, while the shared memory and secure storage, which are dependent on the defined memory size, use 1% of LUTs, 1.8% of LUTRAM, and 0.6% of FFs. TrustShell, despite being central to TREE, only requires a small portion of an entry-level FPGA, according to the results. Consequently, most FPGA logic elements remain available for the TAs loaded on the tiles.

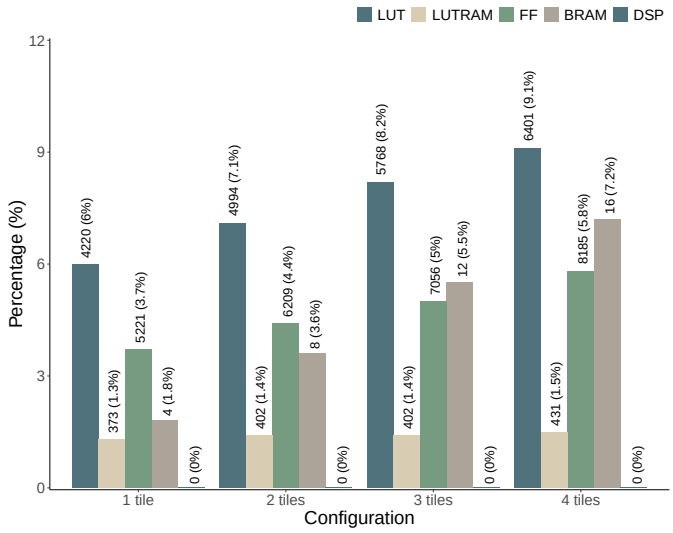


Figure 4: Resource utilization across multiple tile configurations. The plot illustrates how the usage of various FPGA logic elements scales with the increase of tiles, from 1 tile to 4 tiles.

Multi-Tile configuration. As shown in Figure 4, the resource usage for memory logic elements scales predictably with the number of tiles. Each additional tile increases the consumption of logic elements—LUTs, FFs, and BRAMs—by 1.1%, 0.7%, and 1.85%, respectively. LUTRAM usage, however, grows only slightly (<0.1%) since most of the added resources go to shared memory and secure storage, which do not rely on LUTRAM. The tile manager's resource usage remains constant, confirming TREE's scalable design.

6.2 hTAs suppport

The support for hTAs in TREE was evaluated by measuring the execution times of key operations across various tile configurations. Additionally, we assessed the overhead introduced by TREE by comparing the execution time of a hardware storage application with that of a similar native application running independently, with no trusted computing.

Table 3: Microbenchmarks for hTA configuration, clean-up, and TREE-hTA round-trip communication on a tile covering an entire FPGA clock region (954 kiB).

	Config (ms)	**Clean-up (ms)**	**Communication (µs)**
Raw hTA	2.387	2.425	35.5
Encrypted hTA	57.4	2.425	35.5

Table 4: The hardware storage operation reads 8 kiB of data in 1 kiB chunks. The scenarios are: vanilla hardware application configured in the static PL (Native App.) and hTA configuring within TREE.

	Overhead (%)	**Average (ms)**	**Coeff. of Var. (%)**
Native app.	-	21.4 ms	0.05
hTA	0.9	21.6 ms	0.06

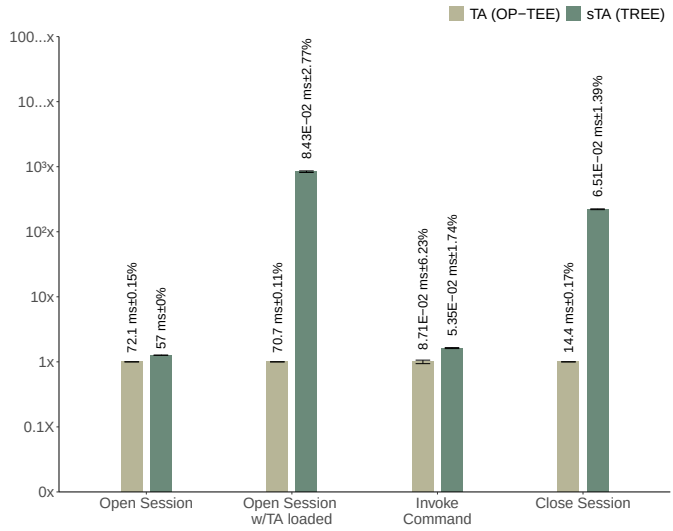


Figure 5: Performance speedup comparison between TREE and vanilla OP-TEE for GP Client API operations: open session, close session, and invoke command.

Methodology. To assess how TREE handles the lifecycle of hTAs, we measured the execution times for hTA configuration, clean-up, and round-trip communication between the CA and TA. Execution time was defined as the duration from the CA's request to the successful return of a status response. The evaluation was conducted using tiles configured to an entire clock region (954 kiB) of the SoC-FPGA.

Micro-operations. Table 3 summarizes the microbenchmark results for the lifecycle of hTA operations on a single tile. The extended configuration time observed for encrypted hTA instances stems from the block-by-block AES decryption process, which could benefit from parallelization optimizations. Once configured, TREE-hTA operations exhibit minimal communication latency, remaining largely unaffected by tile dimensions.

Hardware Storage Application. The results for the hTA overhead are summarized in Table 4. Our evaluation indicates that the performance of hTAs remains comparable to native applications, with an average overhead of less than 1%. This minimal overhead can be attributed to the efficiency of the CA-hTA communication. The results indicate that TREE do not compromise operational efficiency.

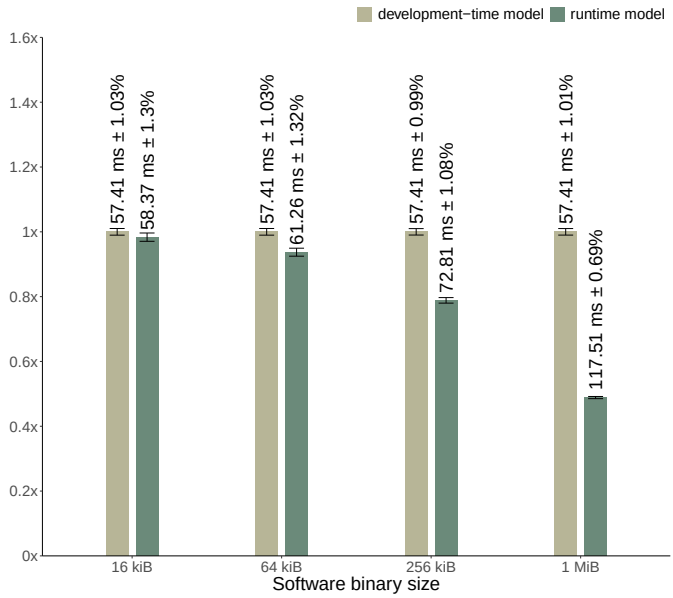


Figure 6: Performance comparison of TA loading models in hybrid TAs. The graph illustrates the relative execution time of the runtime loading model versus the development-time loading model.

6.3 sTAs and GPTAs support

We measure the round-trip communication time for GP-compliant CA operations: opening and closing sessions, and invoking commands. We compared TREE with the vanilla OP-TEE [OP-24], a software-based GP-compliant TEE The goal is to measure the TREE's IPC communication latency while ensuring GP compliance.

Methodology. To enable the execution of software TAs, including GPTAs, TREE must first configure a tile with reference hardware. The reference hardware uses an Arm Cortex-M1 as the soft-core MCU with 64 kiB of BRAM for code and data memory. For the cryptographic operations and Bitcoin TA, we used Linux time API. Due to potential deviations between the performance overhead on an FPGA-based soft MCU at 100 MHz and a hard processor in silicon at 1GHz, we have performed clock frequency normalization. The results of vanilla OP-TEE were normalized by multiplying them by the ratio of its frequency (1 GHz) to the reference of 100 MHz.

Benchmarks. Figure 5 illustrates the execution time comparison between TREE and the vanilla OP-TEE. Opening a session while the TA is already loaded with TREE has a significant speedup compared to OP-TEE, i.e., 83.8x. This improvement is due to TREE's design that treats any subsequent open session as invoked commands, once the TA is loaded. Another significant speedup is regarding the closing session by 22.1x. This improvement is justified by TrustShell's efficient utilization of FPGA parallelism.

6.4 hybridTAs support

TREE provides two models for supporting hybrid TAs: i) *development-time model*; and ii) *runtime model*. Since the primary difference between the models is the software loading mechanism, this evaluation focuses on the performance impact of the TA loading process.

Methodology. We measure the execution time of TA loading operation for both models on the same hybrid application under different conditions. Specifically, we fixed the tile

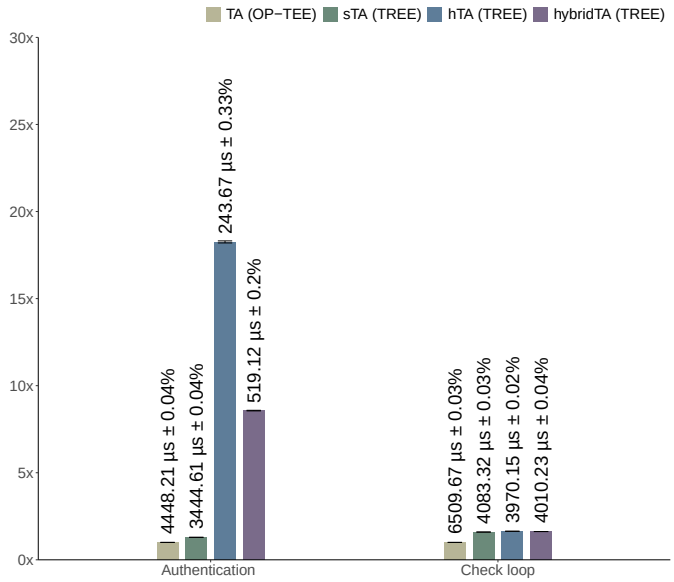


Figure 7: Performance speedup comparison between hardware, software and hybrid versions of the same TA against the same TA running in OP-TEE.

size to one clock region (954 kiB) and varied the software binary size (from 2 kiB, 4 kiB, 8 kiB, up to 16 kiB) to identify the performance trade-offs between the two approaches.

Impact of TA loading. As shown in figure 6, the relative performance of the *runtime model* compared to the *development-time model* decreases as memory size increases. For example, at 16 kiB, the *runtime model* achieves 98.4% of the development-time performance, but this drops to 48.9% at 1 MiB. This is due to the additional time required to load the software after hardware reconfiguration in the *runtime model*, which is more noticeable for larger binaries. For instance, the 1 MiB binary requires an additional 60.1 ms. In contrast, the *development-time model*'s performance is unaffected by binary size, as the software is directly loaded into the BRAM, which serves as on-chip memory for the soft MCU, during partial reconfiguration without requiring additional setup.

7 TREE: Use Cases

7.1 Access control authenticator

The access control authenticator operation is a requirement in a wide range sectors-such as finance, healthcare, and data protection. Its primary goal is to verify user credentials and gate access to protected assets. Given the heterogeneity of workloads, compliance regulations, and security requirements in these industries, developers benefit from customizable solutions. To evaluate these requirements, we developed a minimal access control authenticator in three configurations: i) hardware-only version (hTA); ii) GP-compliant version (sTA); and iii) hybrid version combining both approaches (hybridTA). Each version performs the same core task—digesting new user credentials and comparing them sequentially against a predefined list of authorized credentials. Upon verification, the system returns an authentication result. We also run the same software TA on OP-TEE (TA) as a baseline for comparison.

Methodology: We focused on two key operations: the authentication process (Authentication) and the sequential checking of credentials against the authorized list containing

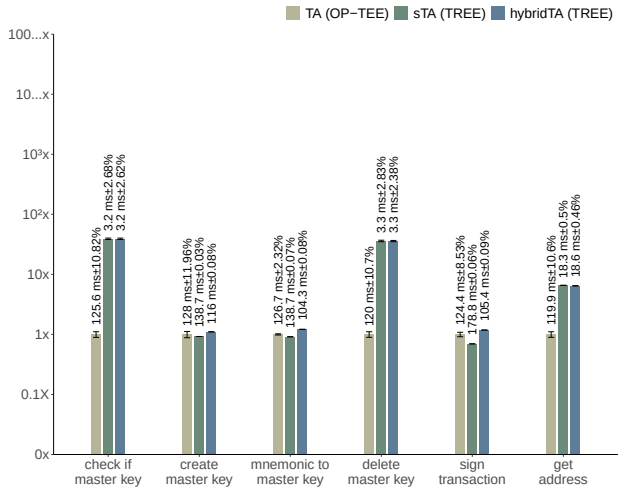


Figure 8: Normalized performance speedup of TREE (vanilla and hybrid versions) and OP-TEE of each Bitcoin wallet operation.

1,024 entries (Check loop). For each version, we measured the round-trip execution time of the invoke command operation calling the respective operation.

Evaluation: Figure 7 presents the performance speedup across the hardware, software, and hybrid implementations compared to the TA running in OP-TEE. The hTA achieved up to 18x speedup for the authentication service, leveraging hardware-accelerated logic. The hybrid version also showed notable speedups but was slower than the hardware-only solution due to its mix of hardware acceleration and software fallback. However, it outperformed the software TA, which achieved a 1.1x speedup. In contrast, for the check loop process, which primarily involves reading and verifying credentials in secure storage, there were no significant performance gains when using the hardware-only or hybrid versions. This is because the check loop is largely bound by read operations rather than intensive calculations, limiting the impact of hardware acceleration.

7.2 Trust Bitcoin wallet

The Bitcoin wallet is a well-fitted real-world example for evaluating TREE, as it handles security-sensitive tasks like key management and transaction signing, which would benefit from the reconfigurability provided by TREE. Since the wallet[2] was initially built for OP-TEE and follows the GP specification, porting it to TREE was straightforward. Both the CA and sTA ran on TREE without any changes. Additionally, we evaluated a modified version of the wallet, in which we migrated the wallet-specific cryptographic algorithms to hardware to assess the performance benefits of TREE's reconfigurable computing.

Methodology: To enable the execution of software TAs, including GPTAs, TREE must first configure a tile with a reference hTA. We use an Arm Cortex-M1 soft-core MCU with 64 kiB of BRAM for code and data. Due to potential deviations between the performance overhead on an FPGA-based soft MCU at 100MHz and a hard processor in silicon at 1 GHz, we have performed clock frequency normalization. We measure the execution time of six operations: i) check if a master key exists; ii) create a new master key; iii) create a new master key from a mnemonic; iv) delete key; v) sign transaction; vi) get public address. Regardless of the operation, CA communicates with the TA by opening a session, invoking a command, and closing the session upon completion.

[2]https://github.com/Jamie-Cui/bitcoin-wallet

Evaluation: Figure 8 presents the results for the Bitcoin wallet TA. Our findings show that the vanilla (sTA) and hybrid (hybridTA) versions running on TREE achieve comparable normalized performance improvements for operations involving simple secure storage verification, such as checking or deleting a master key. This similarity arises because these operations rely only on basic functionalities (e.g., straightforward secure storage access), which do not benefit from hardware acceleration. However, both versions significantly outperform the vanilla version running on OP-TEE, which is justified by the closer access to secure storage and the reduced overhead of GlobalPlatform operation calls. Regarding more intensive cryptographic services (e.g., creating a master key or signing a transaction), the performance gap between the two versions becomes evident. The vanilla version experiences a 1.5x slowdown, while the hybrid version delivers a 1.1x speedup. Additionally, the comparable performance of hybrid TAs and OP-TEE can be explained by the hardware-accelerated cryptographic enhancements, which offset the slower execution of software on a small soft MCU, resulting in overall performance similar to OP-TEE.

8 Security Analysis

8.1 Theoretical security analysis

We analyze TREE's security by theoretically assessing its implementation of security properties. This includes addressing the security requirements and threat vectors detailed in sections 3.2 and 3.1, and emphasizing TREE's protective mechanisms.

Isolate FPGA fabric from the REE (R1). During boot time, access to reconfiguration ports is restricted to maintain a secure boundary between the FPGA fabric and the rest of the SoC, including the APU and peripherals. This restriction is enforced by disabling the external reconfiguration ports (PCAP and JTAG) and allowing only the ICAP to remain active. This ensures that unauthorized software, including potentially compromised applications on the REE OS, cannot reconfigure, read back, clear, or power down the entire or partial FPGA fabric. Consequently, they ensure that TREE modules and loaded TAs remain protected against tampering and malicious interference.

Isolate each TA execution environment (R2). Each TA has its own dedicated execution environment, i.e., a tile, which includes the necessary resources, dedicated access to TEE services (e.g., encryption and secure storage), and essential peripherals for independent execution. Regarding sTAs, this also includes a unique physical address space, CPU, system configuration registers, and TA firmware. To prevent any potential interference or tampering, TAs do not share resources. However, for services like secure storage that cannot be uniquely dedicated to a single TA, TREE does not trust TAs to self-identify. Instead, it uses a backend process that handles the identification of the TA attempting to access it. This approach protects against threats, e.g., resource contention, unauthorized data access, and tampering between TAs. It also protects against attacks where a compromised TA could misuse shared services or alter its own tile's hardware logic to gain unauthorized access or interfere with other TAs.

Ensure TA sensitive data protection (R3). TAs, when idle, remain in the untrusted memory of the REE. Some TAs contain sensitive data, e.g., cryptographic keys, or user credentials, which must be kept confidential even at rest. The TREE framework enables system designers to encrypt their TAs using symmetric cryptography (AES). During secure loading, the encrypted TA is decrypted block by block, with each block sent directly to the ICAP for configuration, avoiding temporary storage of plaintext TAs and reducing exposure risk. Cryptographic keys required for encryption and decryption are securely managed by a hardware security module (HSM), which provides a trusted environment for key generation, storage, and usage. This mechanism protects against threats, e.g., unauthorized memory access, plaintext data exposure, and key extraction attempts.

Table 5: Mitigation analysis of selected CVEs with critical CVSS scores: "✓" - CVEs in scope; "—" - out of scope; "?" - insufficient information.

CVE	C		CVE	C		CVE	C		CVE	C	
2015-4422	TA	✓	2015-9108	TO	✓	2017-18133	TO	✓	2015-8997	TO/TA	✓
2015-6639	TA	✓	2015-9112	TO	✓	2017-18311	TO	✓	2015-8998	TO/TA	✓
2015-6647	TA	✓	2015-9113	TO	✓	2017-18314	TO	?	2015-9005	TO/TA	✓
2015-9000	TA	✓	2015-9198	TO	✓	2017-18315	TO	✓	2015-9007	TO/TA	✓
2015-9002	TA	✓	2015-9199	TO	✓	2017-6290	TO	✓	2016-10297	TO/TA	✓
2015-9162	TA	✓	2015-9200	TO	✓	2017-6292	TO	✓	2016-2432	TO/TA	✓
2015-9174	TA	✓	2016-10238	TO	✓	2017-6294	TO	✓	2017-14913	TO/TA	✓
2015-9183	TA	✓	2016-10239	TO	✓	2017-8274	TO	✓	2017-18293	TO/TA	✓
2017-18310	TA	—	2016-10432	TO	✓	2018-11950	TO	✓	2017-18296	TO/TA	?
2017-18312	TA	?	2016-2431	TO	✓	2018-3588	TO	✓	2017-18297	TO/TA	✓
2017-18317	TA	?	2017-11010	TO	✓	2018-5870	TO	✓	2017-18298	TO/TA	✓
2017-6293	TA	✓	2017-11011	TO	✓	2014-9932	TO/TA	✓	2017-6289	TO/TA	✓
2018-5210	TA	✓	2017-14912	TO	✓	2014-9935	TO/TA	✓	2018-5866	TO/TA	✓
2018-5885	TA	✓	2017-14916	TO	✓	2014-9936	TO/TA	✓	2017-18282	HW	✓
2014-9979	TO	✓	2017-14917	TO	✓	2014-9937	TO/TA	✓	2015-9003	CI	—
2015-8999	TO	✓	2017-17176	TO	✓	2014-9945	TO/TA	✓	2016-10398	CI	—
2015-9070	TO	✓	2017-18071	TO	✓	2014-9948	TO/TA	✓	2017-14907	CI	—
2015-9071	TO	✓	2017-18128	TO	—	2014-9949	TO/TA	✓	2017-18146	CI	—
2015-9072	TO	✓	2017-18129	TO	✓	2015-8995	TO/TA	✓	2016-10458	BL	✓
2015-9073	TO	✓	2017-18132	TO	✓	2015-8996	TO/TA	✓	2017-14911	BL	—

8.2 CVE mitigation analysis

We assess the potential of deploying TREE in real-world systems by analyzing its practical effectiveness in mitigating threats associated with TEE-related vulnerabilities. Our analysis relies on the CVE list compiled by Cerdeira et al. [CMSP22], which provides an extensive collection of TEE-related CVEs. The goal is to understand how TREE's security features, e.g., isolation mechanisms and data protection strategies, directly address these threats. The analysis, summarized in Table 5, focuses on 80 critical CVEs and categorizes the CVEs by the affected components: Trusted OS (TO), Trusted Application (TA), Cryptographic Implementation (CI), Hardware (HW), and Bootloader (BL).

Mitigation of CVEs within scope. TREE addresses 69 of the 80 CVEs analyzed. Most of these vulnerabilities target the trusted OS or TAs, with the potential for privilege escalation attacks. Since TREE meets the security requirement *R2*—isolate each TA execution environment—by ensuring that each TA executes within its own dedicated tile, which includes a unique physical address space, TAs do not share resources with one another nor with the TrustShell, significantly reducing the potential risk of privilege escalation attacks. Additionally, TREE addresses vulnerabilities, e.g., CVE-2016-10458, which exploits improper memory handling between the REE and TEE. By meeting the security requirement *R1*—isolating the FPGA fabric from the REE—TREE ensures a loosely coupled interaction between the REE and TEE, avoiding the need for trusted memory allocation in the same address space as the REE OS.

CVEs out of scope. Out of the 80 CVEs analyzed, 7 CVEs fall outside TREE's scope. For instance, CVE-2017-18310 involves a TA exposing unnecessary services to the REE, potentially allowing malicious actions by CAs. While TREE does not control which services are exposed, it enforces strict isolation between TAs, as required by security requirement *R2*, minimizing the impact of such exposures. TREE also does not address hardware misconfiguration, e.g., CVE-2017-18128, that could result in sensitive data leakage. Since TREE relies on the boot configuration to protect against FPGA readback access to the REE, this vulnerability is also excluded from its scope. Bootloader authentication issues are also unaddressed by TREE, e.g., CVE-2017-14911, as TREE relies on the platform's secure boot process for correct system initialization, it does not address this type of vulnerability either. Cryptographic vulnerabilities, e.g., CVE-2017-18146, are not considered, as the framework assumes the proper implementation of cryptographic algorithms and primitives. However, TREE's flexible architecture, with its FPGA-based TEE, could potentially address these vulnerabilities by enabling over-the-air updates for cryptographic modules, allowing prompt remediation of any weaknesses in cryptographic protocols.

9 Discussion

Encrypted TA key Provisioning. TREE securely loads TAs using an AES symmetric decryption IP to decrypt partial bitstreams before reaching the ICAP, ensuring isolation from the REE and other TAs. While TA key provisioning is not currently supported, our core mechanisms lay the groundwork for its future implementation. Some key exchange protocols necessitate a secure channel for exchange, whereas others like Elliptic Curve Diffie-Hellman (ECDH) do not require a pre-shared secret but should be paired with authentication services. Future iterations will analyze and assess the suitability of these methods for the TREE concept.

SoC-FPGA's main memory management. TREE's threat model assumes DRAM and other SoC-FPGA memories are untrusted. Sharing DDRs among multiple entities (trusted and untrusted) raises significant security concerns particularly side-channel. Adversaries might exploit shared memory access to infer information via timing, power consumption, or electromagnetic emissions, compromising confidentiality and integrity. Future research will analyze the security impacts of allocating some DRAM regions to Tiles and TrustShell services, sharing the DRAM with REE and TEE. This strategic approach aims to enhance scalability and efficiency across the framework.

REE performance impact. Since TREE leverages FPGA solely to provide the execution environment necessary for TAs, it presents a potential to benefit the REE performance as the TEE within the FPGA operates in parallel with the REE. Further research will quantify these performance benefits by analyzing the nature of TAs that would provide the most overall system efficiency. Additionally, this research will determine the suitable use cases for reconfigurable FPGA-based TEEs, identifying scenarios where the unique capabilities of FPGAs can be maximally leveraged.

Tile temporal and spatial multi-Tenancy. TREE treats each tile as a single execution environment, supporting one TA at a time with fixed FPGA space. However, we believe that this could be further enhanced. Some SoC-FPGAs, e.g., AMD's Zynq, support nested DPR, allowing dynamic subdivision of regions. This could overcome the tile size configuration limitation, offering more flexibility in adjusting tile sizes for different hTAs. Additionally, more advanced soft-MCUs with TEE features could be synthesized on TREE, enabling concurrent execution of multiple software TAs and adding an extra security layer. While technically feasible, these multi-tenancy models introduce complexity that may outweigh the benefits for the lightweight TEEs in this research. Exploring these ideas may benefit future systems, but is beyond TREE's current scope.

Physical and side-Channel attacks. FPGA-based attacks, e.g., power analysis, electromagnetic leakage, and fault injection, could potentially compromise the integrity of both the TEE and TAs. TREE does not directly address these specific, indeed intrusive attack scenarios, but many mitigation techniques can be applied. For instance, fault injection resistance can be achieved through redundancy techniques, where a dynamic tile is replicated and the outputs are compared to detect inconsistencies [SZFT23]. Side-channel attack defenses e.g., noise generation, dynamic voltage, and frequency scaling, and power analysis countermeasures can be implemented to mask signals from being exploited in power or electromagnetic analysis [MS19]. Physical probing can be mitigated through techniques, e.g., physical unclonable functions (PUFs) [LC23]. Similar techniques can be considered for future enhancements to TREE, but their mitigation will not impact the current focus on reconfiguration and software isolation security.

Table 6: FPGA-based TEEs state of the art comparison. Key: RoT - Root-of-trust Primitive; dE - Dedicated Execution Enviroment per TA; EC - Execution Enviroment clean-up; ER - Execution Enviroment reuse; TAC - TAs Concurrency; hTAs - Support for hardware TAs; sTAs - Support for software TAs; GPTAs - GP-compliant TAs; SS - Secure Storage; CRYPT - Encryption services; ATT - Verification/attestation services; ●- fares positively; ◐- fares partially positively; ○- fares negatively.

			TEE management				**TA compatibility**			**TEE Services**		
		RoT	dENCLV	EC	ER	TAcurr	hTAs	sTAs	GPTAs	SS	CRYPT	ATT
CPU-driven TEEs	HISA [YFW18]	TrustZone	○	○	○	○	◐	◐	○	●	○	○
	Ambassy [HYea21]	TrustZone	●	○	○	◐	◐	○	○	●	●	●
FPGA TEEs	MeetGo [ONJ+21]	FPGA	●	○	○	○	●	◐	○	○	○	●
	ShEF [ZGK22]	FPGA	●	○	○	○	●	○	○	●	◐	●
Soft MCU TEEs	BYOTee [AZ24]	SoC-FPGA	◐	◐	◐	●	○	●	○	○	○	●
	TEEOD [PCRP21]	SoC-FPGA	●	◐	◐	●	○	●	●	●	●	○
Reconfigurable TEEs	TREE	SoC-FPGA	●	●	●	●	●	●	●	●	●	●

10 Related Work

We compare TREE with other FPGA-based TEE solutions. Table 6 summarizes our findings, focusing on the critical features of TEE design, including the underlying TEE technology, management capabilities, TA compatibility, and TEE services. We also classify these works into three FPGA-based TEE categories: i) extended CPU-driven (CPU-driven TEEs), ii) secure reconfiguration TEEs (FPGA TEEs), and iii) soft MCU-enabled TEEs (Soft MCU TEEs).

Extended CPU-driven TEEs. Well-established processor-built-in TEEs, such as Arm TrustZone [Arm23], have been extended to support reconfigurable computing. Solutions such as HISA [YFW18] and Ambassy [HYea21] use TrustZone capabilities to ensure secure enclave operations. They establish an additional trusted environment on FPGA by leveraging TrustZone's secure world to oversee the secure loading of TAs in bitstream form onto the FPGA. However, this approach inherits inherent microarchitectural vulnerabilities in the underlying hardware's TEE architecture [CSFP20, CMSP22, KHF+19, PAB+18].

Secure reconfiguration TEEs. MeetGo [ONJ+21] and ShEF [ZGK22] propose mechanisms to load custom hardware designs onto a remote FPGA securely and independently of the host system architecture. However, both present limited TA support and lack TEE features such as secure storage. In MeetGo, TA sources are converted to HDL code through high-level synthesis (HLS) tools and mapped onto the FPGA as a partial bitstream, requiring users to translate software applications into hardware logic. In contrast, TREE's secure hardware TA development mechanism eliminates the need for software-to-hardware translation, enabling a more seamless integration for existing applications without compromising interoperability or legacy support for existing TAs.

Soft MCU-enabled TEEs. TEEOD [PCRP21] and BYOTee [AZ24] synthesize soft-core MCUs on an SoC-FPGA to provide secure environments for software TAs to run isolated. TEEOD even complies with the GP TEE specifications. The difference is in the TA cleanup, loading, and management method; BYOTee relies on the firmware running on the soft-MCU and so depends on the enclave's state, while TEEOD offloads these services to dedicated hardware accelerators that are independent of the enclave's state. However, the hardware for executing TAs is allocated at development-time, with no support for dynamic hardware swapping. Any hardware change on the TAs requires rebooting the entire system. As a result, the framework does not support hardware-based TAs, limiting its ability to fully leverage the potential of reconfigurable computing. In contrast, TREE stands out by leveraging DPR technology, commonly found in SoC-FPGAs, to enable loading of TAs' hardware without requiring system reboots, while ensuring their integrity.

11 Conclusion

In this work, we introduce TREE, a novel FPGA-based TEE framework that leverages conventional FPGA mechanisms to enhance current FPGA-based TEEs. Our evaluation reveals minimal overhead, less than 1%, compared to native hardware applications. Additionally, TREE demonstrates significant speedups for TAs optimized for FPGA fabric, while also maintaining compatibility with legacy software TAs. We also demonstrate TREE's ability to handle real-world scenarios, such as access control authentication and Bitcoin wallets without notable performance degradation and, in some cases, speedups.

Acknowledgments

The authors thank the TCHES reviewers and editors for their time and valuable feedback. This work has been supported by FCT – Fundação para a Ciência e Tecnologia within the RD Unit Project of ALGORITMI Centre. The work of Sérgio Pereira was supported by the FCT under Grant No. SFRH/BD/145209/2019. This work is also supported by the European Union's Horizon Europe research and innovation program under grant agreement No 101070537, project CROSSCON (Cross-platform Open Security Stack for Connected Devices).

References

[AMD19] AMD. AMD Secure Encrypted Virtualization, 2019.

[AMD24] AMD. AMD Zynq UltraScale+ MPSoCs Family Reference Guide. https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-ultrascale-plus-mpsoc.html, 2024.

[Arm09] Arm. Security technology building a secure system using TrustZone technology (white paper). https://www.arm.com/technologies/trustzone-for-cortex-a/tee-reference-documentation, 2009.

[Arm23] Arm. TrustZone Technology. https://developer.arm.com/architectures/security-architectures/trustzone, 2023.

[AZ24] M. Armanuzzaman and Z. Zhao. BYOTee: Towards Building Your Own Trusted Execution Environments Using FPGA. In *AsiaCCS*, 2024.

[BBD+21] R. Bahmani, F. Brasser, G. Dessouky, P. Jauernig, Matthias Klimmek, Ahmad-Reza Sadeghi, and Emmanuel Stapf. CURE: A Security Architecture with CUstomizable and Resilient Enclaves. In *Proc. in USENIX Security*, 2021.

[Ben16] G. Beniamini. Extracting Qualcomm's KeyMaster Keys Breaking Android Full Disk Encryption. https://bits-please.blogspot.com/2016/06/extracting-qualcomms-keymaster-keys.html, 2016.

[BMAB17] E. Benhani, C. Marchand, A. Aubert, and L. Bossuet. On the security evaluation of the ARM TrustZone extension in a heterogeneous SoC. In *Proc. in IEEE SOCC*, 2017.

[Car] P. Carru. Attack ARM TrustZone using Rowhammer. https://grehack.fr/data/2017/slides/GreHack17_Attack_TrustZone_with_Rowhammer.pdf.

[CCF+16] Y. Choi, J. Cong, Z. Fang, Y. Hao, G. Reinman, and P. Wei. A quantitative analysis on microarchitectures of modern CPU-FPGA platforms. In *ACM/ EDAC/ IEEE DAC*, 2016.

[CFZ+20] Q. Chen, K. Fan, K. Zhang, H. Wang, H. Li, and Y. Yang. Privacy-preserving searchable encryption in the intelligent edge computing. *Computer Communications*, 2020.

[CMSP22] D. Cerdeira, J. Martins, N. Santos, and S. Pinto. ReZone: Disarming TrustZone with TEE privilege reduction. In *USENIX Security*, Boston, MA, 2022.

[CSFP20] D. Cerdeira, Nuno Santos, Pedro Fonseca, and Sandro Pinto. SoK: Understanding the Prevailing Security Vulnerabilities in TrustZone-assisted TEE Systems. In *Proc. in IEEE S&P*, 2020.

[DWY+22] Y. Deng, C. Wang, S. Yu, S. Liu, Z. Ning, K. Leach, J. Li, S. Yan, Z. He, J. Cao, and F. Zhang. StrongBox: A GPU TEE on Arm Endpoints. ACM, 2022.

[EGEAH+08] T. El-Ghazawi, E. El-Araby, M. Huang, K. Gaj, V. Kindratenko, and D. Buell. The Promise of High-Performance Reconfigurable Computing. *Computer*, 2008.

[Glo18] GlobalPlatform. Introduction to TEEs. https://globalplatform.org/resource-publication/introduction-to-trusted-execution-environments/, 2018.

[Glo24] GlobalPlatform. GlobalPlatform TEE System Architecture, TEE Internal Core API Specification, and TEE Client API Specification. https://globalplatform.org/specs-library/, 2024.

[HBS+08] T. Huffmire, B. Brotherton, T. Sherwood, R. Kastner, T. Levin, T. Nguyen, and C. Irvine. Managing Security in FPGA-Based Embedded Systems. *IEEE Design & Test of Computers*, 2008.

[HYea21] D. Hwang, S. Yeleuov, and et. al. Ambassy: A Runtime Framework to Delegate Trusted Applications in an ARM/FPGA Hybrid System. *IEEE Trans. on Mobile Computing*, 2021.

[Int23] Intel. Intel Software Guard Extensions. https://www.intel.com/content/www/us/en/architecture-and-technology/software-guard-extensions/overview.html, 2023.

[JHZ+17] N. Jacob, J. Heyszl, A. Zankl, C. Rolfes, and G. Sigl. How to break secure boot on fpga socs through malicious hardware. In *CHES*, 2017.

[JTK+19] I. Jang, A. Tang, T. Kim, S. Sethumadhavan, and J. Huh. Heterogeneous Isolated Execution for Commodity GPUs. In *Proc. in ASPLOS*. ACM, 2019.

[K. 19] K. Ryan. Hardware-BackedHeist: Extracting ECDSA Keys from Qualcomm's TrustZone. https://www.nccgroup.trust/globalassets/our-research/us/whitepapers/2019/hardwarebackedhesit.pdf, 2019.

[KHF+19] P. Kocher, J. Horn, A. Fogh, D. Genkin, D. Gruss, W. Haas, M. Hamburg, M. Lipp, S. Mangard, T. Prescher, M. Schwarz, and Y. Yarom. Spectre Attacks: Exploiting Speculative Execution. In *Proc. in IEEE S&P*, 2019.

[LC23] K. Lata and L. Cenkeramaddi. Fpga-based puf designs: A comprehensive review and comparative analysis. *Cryptography*, 2023.

[LFy09] W. Lie and W. Feng-yan. Dynamic Partial Reconfiguration in FPGAs. In *Intelligent Information Technology Application*, 2009.

[LGL+21] R. Liu, L. Garcia, Z. Liu, B. Ou, and M. Srivastava. SecDeep: Secure and Performant On-device Deep Learning Inference Framework for Mobile and IoT Devices. ACM, 2021.

[LSP+17] S. Lee, M. Shih, P., T. Kim, H. Kim, and M. Peinado. Inferring Fine-grained Control Flow Inside SGX Enclaves with Branch Shadowing. In *Proc. in USENIX Security*, 2017.

[MRRL23] A. Muñoz, R. Ríos, R. Román, and J. López. A survey on the (in)security of trusted execution environments. *Computers & Security*, 2023.

[MS19] S. Mirzargar and M. Stojilović. Physical side-channel attacks and covert communication on fpgas: A survey. In *FPL*, 2019.

[NLSY19] H. Ning, Yunfei Li, F. Shi, and L. T. Yang. Heterogeneous edge computing open platforms and tools for internet of things. *Future Generation Computer Systems*, 2019.

[ONJ+21] H. Oh, K. Nam, S. Jeon, Y. Cho, and Y. Paek. MeetGo: A Trusted Execution Environment for Remote Applications on FPGA. *IEEE Access*, 2021.

[OP-24] OP-TEE Documentation Contributors. OP-TEE. https://optee.readthedocs.io/, 2024.

[PAB+18] A. Prout, W. Arcand, D. Bestor, B. Bergeron, C. Byun, V. Gadepally, M. Houle, M. Hubbell, M. Jones, A. Klein, P. Michaleas, L. Milechin, J. Mullen, A. Rosa, S. Samsi, C. Yee, A. Reuther, and J. Kepner. Measuring the Impact of Spectre and Meltdown. In *Proc. in IEEE HPEC*, 2018.

[PCRP21] S. Pereira, David Cerdeira, Cristiano Rodrigues, and Sandro Pinto. Towards a Trusted Execution Environment via Reconfigurable FPGA, 2021.

[Pet24] E. Peterson. Developing tamper-resistant designs with ultrascale and ultrascale+ FPGAs. https://www.xilinx.com/support/documentation/application_notes/xapp1098-tamper-resist-designs.pdf, 2024.

[PPM15] J. Park, Y. Park, and S. Mahlke. Chimera: Collaborative Preemption for Multitasking on a Shared GPU. *Proc. in ASPLOS*, 2015.

[RMIH23] P. Rosero-Montalvo, Z. István, and W. Hernandez. A Survey of Trusted Computing Solutions Using FPGAs. *IEEE Access*, 2023.

[SAB15] M. Sabt, M. Achemlal, and A. Bouabdallah. Trusted Execution Environment: What It is, and What It is Not. In *IEEE Trustcom/ BigDataSE/ ISPA*, 2015.

[SCG+03] G.E. Suh, D. Clarke, B. Gasend, M. van Dijk, and S. Devadas. Efficient memory integrity verification and encryption for secure processors. In *Proc. in IEEE/ACM MICRO*, 2003.

[SWGI+16] R. Shu, P. Wang, S. A Gorski III, B. Andow, A. Nadkarni, L. Deshotels, J. Gionta, W. Enck, and X. Gu. A study of security isolation techniques. *Comput. Surv.*, 2016.

[SZFT23] A. M. Shuvo, T. Zhang, F. Farahmandi, and M. Tehranipoor. A comprehensive survey on non-invasive fault injection attacks. Cryptology ePrint Archive, 2023.

[Tay17] B. Taylor. *"State-of-the-Art Programmable Logic"*, pages 1–15. Springer International Publishing, 2017.

[TM14] S. Trimberger and J. Moore. FPGA Security: Motivations, Features, and Applications. *IEEE Access*, 2014.

[Tri94] S. Trimberger. *Field-Programmable Gate Array Technology*. Kluwer, 1994.

[TSS17] A. Tang, S. Sethumadhavan, and S. Stolfo. CLKSCREW: Exposing the Perils of Security-Oblivious Energy Management. In *Proc. in USENIX Security*, 2017.

[VF18] K. Vipin and S. A. Fahmy. FPGA Dynamic and Partial Reconfiguration: A Survey of Architectures, Methods, and Applications. *ACM Comput. Surv.*, 2018.

[VVB18] S. Volos, K. Vaswani, and Rodrigo Bruno. Graviton: Trusted Execution Environments on GPUs. In *USENIX OSDI*, 2018.

[Xil24] AMD Xilinx. MicroBlaze Soft Processor Core. https://www.xilinx.com/products/design-tools/microblaze.html, 2024.

[XLXW21] K. Xia, Y. Luo, X. Xu, and S. Wei. SGX-FPGA: Trusted Execution Environment for CPU-FPGA Heterogeneous Architecture. In *ACM/ IEEE DAC*, 2021.

[YFW18] M. Ye, X. Feng, and S. Wei. HISA: Hardware Isolation-based Secure Architecture for CPU-FPGA Embedded Systems. In *IEEE/ACM ICCAD*, pages 1–8, 2018.

[ZGK22] M. Zhao, M. Gao, and C. Kozyrakis. ShEF: Shielded Enclaves for Cloud FPGAs. In *Proc. in ACM ASPLOS*, 2022.

[ZHW+20] J. Zhu, R. Hou, X. Wang, W. Wang, Jiangfeng Cao, Boyan Zhao, Zhongpu Wang, Yuhui Zhang, Jiameng Ying, Lixin Zhang, and Dan Meng. Enabling Rack-scale Confidential Computing using Heterogeneous Trusted Execution Environment. In *Proc. in IEEE S&P*, 2020.